REDES DE COMPUTADORAS · TEC · II SEMESTRE 2026

Explicación del proyecto FaroNet (FaSolis)

Todo el repositorio explicado desde cero, capa por capa, como si nunca hubieras visto una red. Con énfasis en el enrutamiento por **cebolla** y, sobre todo, en la parte de **Cloudflare**.

1 · El Faro
Luz infrarroja, tramas, CRC y ACK.

2 · La mesh y la cebolla
Rutas, llaves, directorio y anonimato en 3 saltos.

3 · El edge (Cloudflare)
Worker, Durable Objects, salas, feeds, buzón y directorio público.

Integrantes del proyecto: Joshua Solís Fuentes y Jesús Andrey Solís Mora · Profesor: Kevin Moraga
Documento de estudio generado a partir del código del repositorio (estado del 26 de octubre de 2026).

Contenido

Parte 0 · Antes de empezar
Cómo leer este documento y el mapa del repositorio.

Parte 1 · Conceptos desde cero
Red, paquete, capas, bytes, hash, cifrado, firma digital, metadatos.

Parte 2 · La idea general de FaroNet
Las tres zonas y por dónde viaja un mensaje.

Parte 3 · Capa 1: el Faro
Luz infrarroja, puerto serie, la trama, CRC, ACK, fragmentos, LEDs y botón de pánico.

Parte 4 · La red mesh
El paquete de 512 bytes, el nodo, vecinos, rutas tipo RIP, identidad y directorio.

Parte 5 · La cebolla (énfasis)
Intuición, criptografía paso a paso, formato de la celda, qué ve cada relay, relleno determinista, tipos de mensaje, rotación de llaves y limitaciones.

Parte 6 · Cloudflare (énfasis principal)
Qué es Cloudflare, edge, serverless, Workers, Durable Objects, WebSockets, alarmas, Wrangler; luego cada archivo del Worker, el gateway, el flujo de punta a punta, seguridad web, privacidad, pruebas y despliegue.

Parte 7 · Pruebas automáticas
Qué se prueba y cómo correrlo.

Parte 8 · Observaciones del repositorio
Detalles desactualizados o faltantes que conviene revisar.

Parte 9 · Preguntas que te pueden hacer
Con respuestas cortas, para la revisión.

Parte 10 · Glosario
Todos los términos en un solo lugar.

Parte 0 · Antes de empezar

FaroNet es una red de mensajería que intenta proteger no solo **lo que dices**, sino también **quién lo dice y a quién**. Para lograrlo combina tres cosas muy distintas: un aparato de luz soldado a mano, una red de computadoras que se pasan mensajes entre sí, y unas funciones que corren en los servidores de Cloudflare.

El documento va de lo más básico a lo más específico. Si algo de las partes 5 o 6 no se entiende, casi siempre la explicación está en la Parte 1. Los nombres de archivos aparecen en `este formato` para que los busques en el repositorio.

Mapa del repositorio

Carpeta / archivo	Qué contiene	Lenguaje
`faronet/faro/`	Capa 1: la trama de luz (`capa1.py`), el Faro como vecino de la red (`enlace_faro.py`) y los LEDs y botón de pánico (`hardware.py`).	Python
`faronet/mesh/`	La red anónima: formato de paquete, llaves, directorio, **cebolla** y el nodo.	Python
`faronet/herramientas/`	Programas de terminal: enviar/recibir por luz, medir el enlace, nodo con Faro, **gateway** y cliente del **buzón**.	Python
`edge/`	**El Worker de Cloudflare**: código en `edge/src/` , configuración en `wrangler.jsonc` , pruebas en `edge/pruebas/` .	TypeScript
`pruebas/`	84 pruebas automáticas de la parte Python.	Python
`scripts/probar_cebolla.ps1`	Abre en Windows un directorio y 6 nodos encadenados para probar la cebolla.	PowerShell
`docs/`	El Kick-off y el diagrama de arquitectura.	Markdown

Cómo dependen los módulos entre sí (cada uno solo usa los de su derecha): `faro_nodo → nodo → cebolla → directorio → identidad → paquete` , y por el lado del hardware `enlace_faro / hardware → capa1` . El gateway agrega `gateway_nodo → gateway → (HTTP) → edge` .

PARTE 1

Conceptos desde cero

Estas ideas aparecen en todo el proyecto. Vale la pena entenderlas bien una vez.

1.1 ¿Qué es una red y qué es un paquete?

Una **red** es un grupo de máquinas que se pueden mandar datos. Los datos no viajan como un chorro continuo, sino partidos en pedazos llamados **paquetes**. Cada paquete lleva una parte de **encabezado** (información para entregarlo: a quién va, de qué tipo es, cuánto mide) y una parte de **contenido** (lo que realmente se quiere enviar).

> **Analogía**
>
> Un paquete es como una carta: el sobre (encabezado) dice a dónde va; la hoja de adentro (contenido) es el mensaje. El cartero solo necesita leer el sobre.

1.2 Las capas

Las redes se diseñan **por capas**. Cada capa resuelve un problema y le ofrece un servicio a la de arriba, sin que la de arriba tenga que saber cómo lo hace. En FaroNet las capas son:

Aplicación pública (edge)	Cloudflare Worker: salas, feeds, buzón, directorio público
Privacidad (cebolla)	cebolla.py: 3 relays, cifrado por capas
Capas 2 y 3 (mesh)	nodo.py, paquete.py: vecinos, rutas, reenvío
Capa 1 (física)	capa1.py: tramas por luz · o TCP/IP normal (Ethernet/WiFi)
Medio: luz infrarroja (Faro) o cable/aire (Ethernet, WiFi)	

Cada capa usa la de abajo como una "caja negra". Al nodo le da igual si un vecino está conectado por TCP o por luz.

1.3 Bytes, hexadecimal y tamaños fijos

Un **byte** son 8 bits (8 ceros o unos); puede valer de 0 a 255. Para escribir bytes de forma corta se usa **hexadecimal**: cada byte son dos símbolos de `0-9` y `a-f`. Por ejemplo `AA 55` son dos bytes. Una llave de 32 bytes se escribe con 64 caracteres hex (por eso `GATEWAY_PUBKEY` mide 64 caracteres).

FaroNet usa mucho los **tamaños fijos**: todos los paquetes de la mesh miden 512 bytes, todas las celdas de la cebolla 487, todos los blobs del buzón 1024. ¿Por qué? Porque **el tamaño también es información**. Si un mensaje "sí" mide menos que uno largo, un espía aprende algo sin leerlo. Con tamaño fijo, todos se ven iguales. Lo que sobra se llena con **relleno** (padding).

1.4 Hash

Un **hash** es una función que convierte cualquier dato en una "huella" corta de tamaño fijo. Cambiar un solo bit del dato cambia toda la huella, y no se puede volver de la huella al dato. FaroNet usa **SHA-256** (en el

edge) y **BLAKE2b** (en Python). Un hash **con llave** (keyed hash) solo lo puede calcular quien tenga la llave: eso es un **MAC** (código de autenticación de mensaje), y sirve para detectar si alguien alteró algo.

1.5 Cifrado simétrico y asimétrico

> **Simétrico**
> La misma llave cifra y descifra. Rápido. Problema: ¿cómo se ponen de acuerdo los dos en la llave? En el proyecto: `SecretBox` (XSalsa20 + Poly1305).

> **Asimétrico (llave pública)**
> Cada quien tiene una **llave privada** (secreta) y una **pública** (la puede ver todo el mundo). Lo cifrado hacia tu pública solo lo abre tu privada. En el proyecto: **X25519**, `SealedBox` y `Box`.

Diffie-Hellman (DH) es el truco que une las dos ideas: si yo tengo la privada *a* y tú la pública *B*, y tú tienes la privada *b* y mi pública *A*, los dos podemos calcular **el mismo secreto** sin habérnoslo enviado nunca. Ese secreto luego se usa como llave simétrica. X25519 es una forma moderna de hacer DH. Este truco es el corazón de la cebolla.

Una **llave efímera** es una llave que se crea para un solo uso y se tira. Si alguien la roba después, no le sirve de nada.

1.6 Firma digital

Con una **llave de firma privada** se "firma" un dato; con la **pública** correspondiente cualquiera puede comprobar que (1) lo firmó el dueño de esa llave y (2) nadie lo cambió después. No oculta el contenido, solo lo autentica. El proyecto usa **Ed25519** en dos lugares: las fichas del directorio (cada nodo firma la suya) y las peticiones del gateway hacia Cloudflare.

1.7 Metadatos: el problema que FaroNet ataca

Aunque un mensaje vaya cifrado, quedan los **metadatos**: quién habló con quién, cuándo, desde dónde y cuánto. Con eso basta para vigilar a alguien. Por eso el proyecto no se conforma con cifrar: también oculta el origen (cebolla), iguala los tamaños (relleno), no guarda IPs ni bitácoras, y borra todo lo que vence.

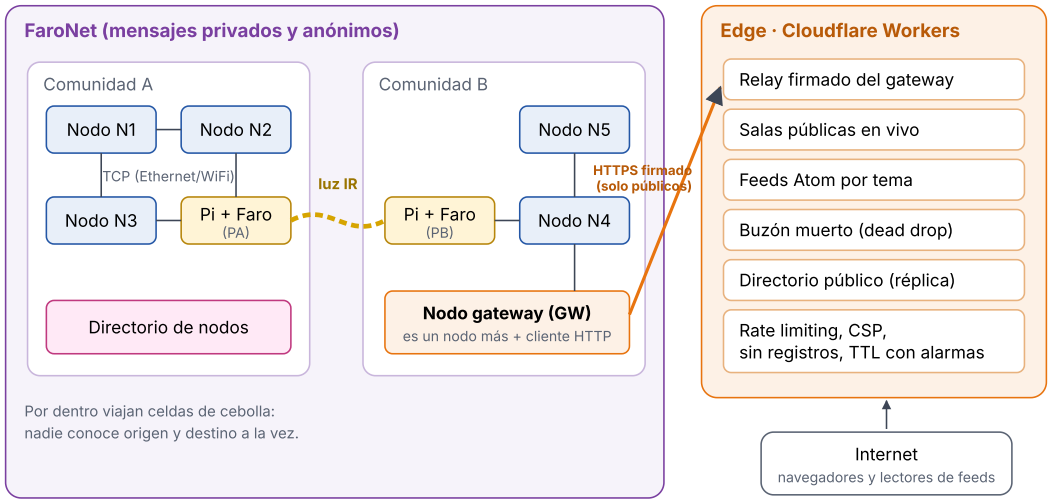
1.8 Ataques que el código tiene en cuenta

Ataque	En qué consiste	Defensa en FaroNet
Repetición (replay)	Capturar un mensaje válido y volver a mandarlo tal cual.	Recordar lo ya visto (llaves efímeras en la cebolla, firmas en el edge) y marcas de tiempo.
Suplantación	Hacerse pasar por otro nodo.	Fichas firmadas; un nombre no cambia de llave mientras su ficha viva.
Alteración	Cambiar bits de un mensaje en camino.	CRC (errores de la luz), MAC por salto (cebolla), firmas.
Correlación	Deducir quién habla con quién por tamaños o tiempos.	Tamaños fijos. Los tiempos no se ocultan (limitación documentada).
Abuso / saturación	Inundar un servicio de peticiones.	Límite de peticiones (rate limiting) en el edge; topes de memoria en todo.

PARTE 2

La idea general de FaroNet

El sistema se divide en dos mundos: **FaroNet**, la red privada donde viven los mensajes anónimos, y el **edge** (Cloudflare), la ventana pública a Internet. Entre los dos hay una sola puerta: el **gateway**, que solo deja salir los mensajes marcados como públicos.



Vista simplificada. El diagrama original del equipo está en `docs/DiagramaModelo.png`.

Las piezas, en una línea cada una

- **Nodo**: cada participante corre `nodo.py` en una laptop o Raspberry Pi. Se conecta a vecinos, aprende rutas y reenvía paquetes.
- **Faro**: un circuito de luz infrarroja conectado a una Raspberry Pi. Une *una comunidad con otra* por luz. Para la red es simplemente un vecino más.
- **Directorio de nodos**: un servicio donde cada nodo publica su "ficha" firmada (nombre + llaves públicas), para que otros puedan cifrarle.
- **Cebolla**: la forma de mandar un mensaje pasando por 3 nodos al azar, envuelto en 3 capas de cifrado, para que nadie sepa a la vez quién lo mandó y a quién.
- **Gateway**: un nodo especial que recibe los mensajes *públicos* por la cebolla y los publica en Cloudflare con una petición firmada.
- **Edge (Cloudflare Worker)**: código que corre en los servidores de Cloudflare y muestra al mundo las salas, los feeds, un buzón cifrado y una copia del directorio.

Tres formas de mandar un mensaje

Comando	Qué pasa	Quién sabe qué
`B hola`	Mensaje directo por la mesh (sin cebolla). El paquete lleva el destino pero no el origen.	Los nodos de en medio ven el destino y el texto.
`anonimo B hola` `firmado B hola`	Por la cebolla, 3 relays. "Firmado" mete tu nombre *dentro* del sobre sellado.	Relays: nada útil. B: el texto (y tu nombre si firmaste).
`publico GW general hola`	Por la cebolla hasta el gateway, que lo publica en la sala `#general` de Cloudflare.	Gateway y mundo: el texto y el tema. Nadie: quién lo escribió.

PARTE 3

Capa 1: el Faro

El Faro es el “cable” hecho de luz. Un LED infrarrojo prende y apaga muy rápido para mandar bits; un fototransistor del otro lado los detecta. La Raspberry Pi controla todo por su **puerto serie** (UART), así que el Faro no necesita un microcontrolador propio.

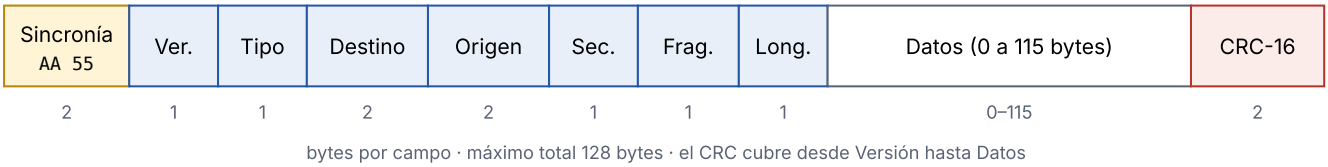
3.1 El hardware

- **Emisor**: LED infrarrojo + resistencia, activado por un transistor (la Pi sola no da suficiente corriente).
- **Receptor**: fototransistor + resistencia: convierte la luz que recibe en un voltaje que la Pi lee.
- **Indicadores**: LED verde de transmisión (GPIO17) y LED amarillo de recepción (GPIO27).
- **Botón de pánico** (GPIO22, con resistencia pull-up: presionado lee 0).

El puerto serie manda cada byte como 10 bits (1 de inicio, 8 de datos, 1 de parada). La velocidad se mide en **baudios**. El código menciona 100 baudios como el enlace actual y prueba también 1200 y 2400. A 100 baudios salen unos 10 bytes por segundo: la luz es **muy lenta** comparada con WiFi, y por eso el diseño es tan cuidadoso con reintentos y tiempos.

3.2 La trama (`capa1.py`)

Por la luz no se mandan bytes sueltos, sino **tramas**: bloques con un formato fijo de máximo 128 bytes.



bytes por campo · máximo total 128 bytes · el CRC cubre desde Versión hasta Datos

Campo	Para qué sirve
Sincronía `AA 55`	Marca “aquí empieza una trama”. El receptor busca este patrón entre la basura que pueda llegar por la luz.
Tipo	1 = datos, 2 = ACK (confirmación), 3 = trama “externa” (creada fuera del Faro; el Faro solo la transporta).
Destino / Origen	**Direcciones físicas** de 2 bytes, al azar. `FFFF` = difusión (para todos, sin ACK). Se guardan en `~/.faronet_direccion` y se pueden rotar.
Secuencia	Número de la trama, para emparejar cada ACK con su trama y detectar duplicados.
Fragmento	Bit 7 = “vienen más”; bits 0-6 = número de pedazo. Permite mandar mensajes de hasta 128 × 115 bytes.
CRC-16	Suma de verificación: si un bit se daña en el camino, el CRC no cuadra y la trama se descarta. Detecta errores *accidentales*, no ataques.

3.3 Cómo se reciben tramas entre ruido

El `Decodificador` recibe bytes sueltos y entrega solo tramas válidas. Su “regla de oro”: si algo no cuadra (versión rara, longitud absurda, CRC malo), **no tira todo**; descarta solo el primer byte y vuelve a buscar `AA 55` . Así, si dentro de lo dañado venía el inicio de una trama buena, la rescata. Si dejan de llegar bytes a mitad de trama, la abandona (`expirar()`).

3.4 Acceso al medio: maestro, esclavo y ACK

Solo hay un "canal" de luz, así que hay que turnarse. `DispositivoLuzAdaptador` usa un esquema **maestro/esclavo**:

1. Quien tiene algo que enviar actúa de **maestro**: manda la trama y espera un **ACK** con la misma secuencia.
2. El otro, de **esclavo**, al recibir una trama válida para él responde con un ACK (13 bytes, sin datos). Si es un duplicado (se perdió el ACK y el maestro reintentó), vuelve a confirmar pero no la entrega dos veces.
3. Si el ACK no llega a tiempo, el maestro **reintenta** (3 veces por defecto). Todos los tiempos se calculan a partir de los baudios, para que funcione igual a 100 que a 2400.
4. Mensajes largos se **fragmentan** en tramas de 115 bytes, cada una con su ACK; el receptor las reensambla.

> **Cuenta rápida**
> Un paquete de la mesh mide 512 bytes → 5 fragmentos (4 de 115 y 1 de 52). Con encabezados y ACKs son unos 640 bytes por la luz. A 100 baudios eso es más de un minuto; a 2400 baudios, unos 3 segundos. Por eso, cuando hay Faro, se recomienda `--intervalo-rutas 20` : anunciar rutas cada 5 s saturaría la luz.

3.5 El Faro como vecino de la mesh (`enlace_faro.py`)

El nodo habla con sus vecinos usando un *reader* y un *writer* de asyncio (lo normal para TCP). `EnlaceFaro` le da al nodo un par que **imita** esa interfaz, pero por debajo manda los paquetes de 512 bytes por luz. Como el adaptador bloquea (espera, duerme), vive en **su propio hilo**. Ese hilo: si hay algo en la cola y el medio está libre, envía como maestro; si falla, espera un tiempo **aleatorio** (para que los dos Faros no choquen siempre a la vez); si no hay nada, escucha como esclavo. La dirección del otro Faro se aprende sola: se envía a difusión hasta recibir el primer paquete.

3.6 LEDs, botón de pánico y medición

- `hardware.py` : `Indicadores` prende el LED verde al escribir y el amarillo al leer. En una laptop sin GPIO no falla, simplemente no hay LEDs. `python3 -m faronet.faro.hardware` hace una autoprueba del circuito.
- **Botón de pánico** (`ejecutar_panico`): detiene el nodo, **sobrescribe con ceros y borra** los archivos de llaves, **rota la dirección física** y lo indica con los LEDs. Limitación honesta documentada: en una microSD, el controlador puede escribir los ceros en otro bloque y la copia vieja sobrevivir; guardar las llaves en RAM (`/dev/shm`) lo evitaría.
- `medir_enlace.py` : mide el **BER** (porcentaje de bits errados, mandando bytes crudos conocidos) y la **tasa efectiva** (bytes útiles por segundo con todo el protocolo). Imprime una línea `RESULTADO;...` para la tabla del informe.
- `enviar.py` / `recibir.py` : mandar y recibir texto o archivos por el Faro, para pruebas.
- `faro_nodo.py` : el nodo completo para la Pi: `nodo.py` + el otro Faro como vecino + LEDs + botón. Agrega los comandos `faro` y `rotar` .

PARTE 4

La red mesh

Una red **mesh** (malla) no tiene un centro: cada nodo se conecta con unos pocos vecinos y entre todos se pasan los paquetes hasta que llegan. Esta parte explica cómo un nodo sabe por dónde mandar algo.

4.1 El paquete de 512 bytes (`paquete.py`)



Como todos miden 512, sobre TCP basta con leer exactamente 512 bytes por paquete: no hace falta separador. “Longitud” dice cuántos bytes del contenido son reales; el resto es basura aleatoria. Los tipos son:

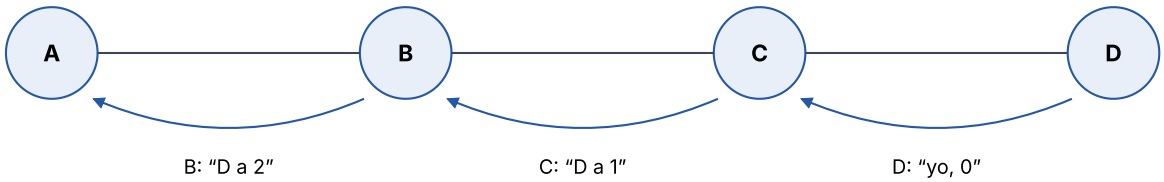
Tipo	Nombre	Uso
1	HOLA	Saludo entre vecinos: “me llamo B”. Lleva una bandera “contéstame” para que el saludo sobreviva si se perdió por la luz.
2	DATOS	Mensaje directo (sin cebolla): TTL | destino | texto.
3	RUTAS	Anuncio del vector de distancias: lista de (nombre, distancia).
4	CEBOLLA	Una celda de cebolla viajando al siguiente relay.
5	MENSAJE	El mensaje sellado que sale del último relay hacia el destino.
10–14	DIR_*	Conversación con el directorio: publicar, consultar, ficha, fin, respuesta.

4.2 El nodo (`nodo.py`)

Cada máquina corre un nodo. Al arrancar: escucha en un puerto TCP, se conecta a los vecinos que se le pasen con `--vecino host:puerto` (reintentando cada 2 s), saluda con HOLA, y desde ahí intercambia rutas y paquetes. También publica su ficha en el directorio cada 60 s y ofrece una terminal con comandos (`anonimo` , `firmado` , `publico` , `rutas` , `directorio` , `rotarllaves` , `vecinos` , `salir`).

4.3 ¿Cómo aprende las rutas? Vector de distancias (tipo RIP)

Ningún nodo conoce el mapa completo. Cada 5 segundos, cada nodo le dice a sus vecinos: “yo llego a X en tantos saltos”. Quien lo escucha suma 1 y se queda con el camino más corto. Eso es el algoritmo de **Bellman-Ford** distribuido, el mismo de RIP.



A anota: D a 3 saltos, vía B

- **Infinito = 16**: una distancia de 16 significa “inalcanzable” (como en RIP).
- **Envenenamiento en reversa**: a un vecino se le anuncian como infinito las rutas que pasan por él mismo. Así nunca intenta llegar a algo “devolviéndose” por nosotros, lo que evita bucles.
- **Actualización inmediata**: si algo cambia, se anuncia de una vez, sin esperar los 5 s.
- **Envejecimiento**: una ruta sin noticias durante 3 intervalos se marca infinita y luego se borra. Si un vecino se desconecta, todo lo que pasaba por él se vuelve infinito.
- **TTL = 16**: cada paquete lleva un contador que baja en cada salto; en 1 se descarta. Evita que un paquete gire para siempre si hay un bucle.

> **Privacidad desde el diseño**
> El paquete de datos lleva el **destino** pero **no el origen**: quien lo recibe no sabe de qué nodo salió. Los paquetes CEBOLLA y MENSAJE llevan el destino siempre en 16 bytes fijos y la carga siempre del mismo largo. El nodo no imprime ni guarda IPs.

4.4 Identidad: dos pares de llaves (`identidad.py`)

Llave	Para qué	Dónde vive	¿Rota?
Firma (Ed25519)	Demostrar que una ficha la publicó ese nodo.	En disco: `~/.faronet/<nombre>.llaves` (32 bytes, permisos 600). Es lo que borra el botón de pánico.	No (larga vida).
Cifrado (X25519)	Que otros le manden datos que solo él lee (la usa la cebolla).	**Solo en memoria.** Nunca toca el disco.	Sí: cada hora por defecto; la vieja se acepta 10 min más (“gracia”) y luego se olvida.

La **ficha** es la tarjeta de presentación de un nodo:

```
Versión(1) | LongNombre(1) | Nombre | LlaveFirma(32) | LlaveCifrado(32) | Expira(8) | Firma(64)
             └──────────── todo esto lo cubre la firma Ed25519 ────────────┘
```

Una ficha nueva vale 5 minutos y se vuelve a publicar cada minuto (o de inmediato tras rotar llaves). La **huella** es un resumen corto de la llave de firma, para comparar a simple vista.

4.5 El directorio de nodos (`directorio.py`)

Un servicio TCP donde los nodos publican su ficha y cualquiera consulta todas. Reglas al publicar: firma válida; no vencida; vencimiento a lo sumo 1 hora adelante; si el nombre ya existe con **otra** llave de firma y esa ficha sigue viva, se rechaza (anti-suplantación); una ficha más vieja que la registrada se rechaza (anti-repetición). Todo vive en memoria, sin IPs ni bitácoras. Lo más importante: **los nodos no confían en el directorio**, verifican cada firma ellos mismos. El directorio puede esconder fichas, pero no inventarlas.

PARTE 5 · ÉNFASIS

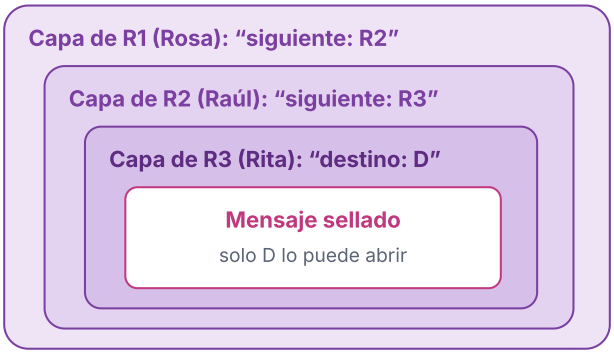
La cebolla (`cebolla.py`)

El objetivo de la cebolla es uno solo: que **ningún nodo intermedio conozca a la vez el origen y el destino** de un mensaje. Es la misma idea de Tor, simplificada y adaptada a paquetes de 512 bytes.

5.1 La intuición: sobres dentro de sobres

Imagina que quieres mandarle una carta a Daniela sin que nadie sepa que fuiste tú. Escoges a tres mensajeros al azar: Rosa, Raúl y Rita.

1. Escribes la carta y la metes en un sobre que **solo Daniela** puede abrir.
2. Ese sobre lo metes en otro que **solo Rita** puede abrir, con una nota: "entrégalo a Daniela".
3. Eso lo metes en otro que **solo Raúl** puede abrir: "entrégalo a Rita".
4. Y eso en otro que **solo Rosa** puede abrir: "entrégalo a Raúl". Se lo das a Rosa.



Cada relay, al recibir:

1. Quita su capa (solo él tiene la llave).
2. Lee únicamente a quién se lo pasa.
3. Reenvía lo que queda, sin poder mirarlo.

Resultado:

R1 sabe de quién lo recibió, pero no el destino.
R3 sabe el destino, pero no el origen.
Nadie salvo D puede leer el mensaje.

Cada capa es como la capa de una cebolla: se va pelando una en cada salto. Lo difícil (y lo que hace el código) es lograr esto con matemáticas en vez de sobres, que la "cebolla" **mida siempre lo mismo** en cada salto, y que nadie pueda alterarla sin que se note.

5.2 ¿Qué ve cada participante?

Quién	Ve	NO ve
Emisor	Todo (él armó la cebolla).	—
R1 (primer relay)	El vecino que se lo pasó y que el siguiente es R2.	Si ese vecino es el emisor o solo un intermediario; el destino; el texto.
R2 (relay del medio)	Que viene de R1 y va a R3.	Emisor, destino, texto. Es el que menos sabe.
R3 (último relay)	Que viene de R2 y el **destino** final.	El emisor y el texto (va sellado para el destino).
Destino	El texto. Si el mensaje es "firmado", también el nombre verificado del emisor.	En modo anónimo, quién lo envió.

Nodos de la mesh entre relays	Solo “este paquete va hacia R2” (por ejemplo).	Todo lo demás: la celda va cifrada.

> **¿Por qué 3 relays?**
> Con 1 relay, ese relay vería origen y destino. Con 2, el primero conoce el origen y el segundo el destino, y si se ponen de acuerdo (o son el mismo dueño) lo saben todo. Con 3, el del medio separa a los otros dos: hacen falta los tres coludidos. Es la misma elección que Tor. En el código es la constante `SALTOS = 3` y define todos los tamaños. Por eso **hacen falta al menos 5 nodos**: emisor + 3 relays + destino (los relays nunca son ni el emisor ni el destino).

5.3 Los ingredientes criptográficos

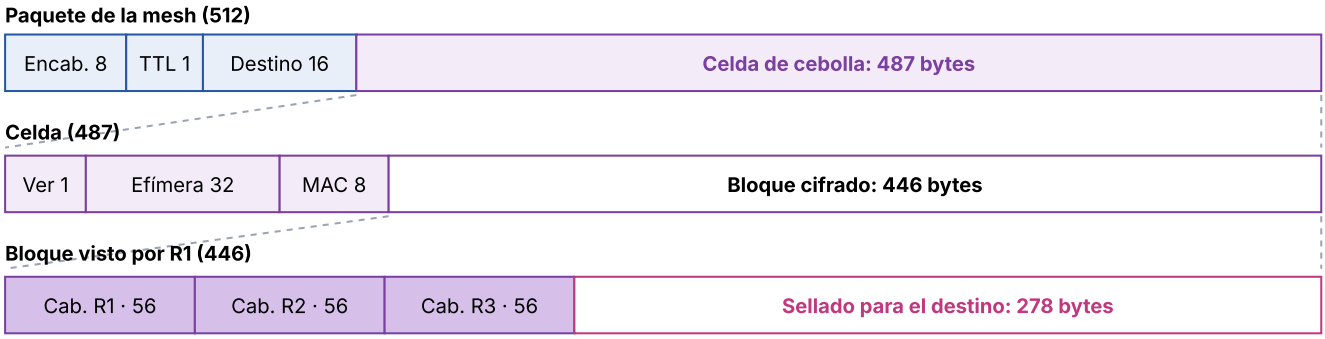
Todo usa **PyNaCl** (la biblioteca libsodium). El proyecto no inventa primitivas criptográficas: solo combina piezas probadas.

Pieza	Qué hace en la cebolla
Llave efímera X25519 por relay	Para cada mensaje y cada relay, el emisor crea un par de llaves nuevo. La pública viaja en la celda; la privada se tira tras armar la celda.
DH(efímera, llave de cifrado del relay)	Da un secreto que solo el emisor y ese relay pueden calcular. El emisor saca la pública del relay de su **ficha del directorio**.
`_derivar()` con BLAKE2b	Mezcla `versión + secreto + efímera + llave del relay` con la etiqueta `"faronet-cebolla-v1"` y saca 64 bytes: los primeros 32 son la **llave de flujo**, los otros 32 la **llave de MAC**.
`_flujo()` (keystream)	Con la llave de flujo genera una tira de bytes pseudoaleatorios (cifrando ceros con `SecretBox` /XSalsa20). Cifrar = hacer XOR del bloque con esa tira. Un cifrado de flujo **no cambia el tamaño**: clave para que la celda mida siempre igual.
XOR	Operación bit a bit: A XOR K = C, y C XOR K = A. La misma operación cifra y descifra.
MAC por salto (8 bytes)	Un hash con llave sobre **todo el bloque** que recibe ese relay. Si alguien altera un solo bit, el siguiente relay honesto lo detecta y descarta la celda.
`SealedBox`	Cifra el mensaje final hacia la llave pública del **destino**: extremo a extremo; ni R3 lo abre.
`Box` (solo en “firmado”)	Cifra con la llave del emisor hacia la del destino: el destino sabe que solo ese emisor pudo crearlo.

> **Un detalle que suele preguntarse**
> `_flujo()` usa un nonce fijo (24 ceros). Normalmente eso sería un error grave, pero aquí cada llave de flujo sale de una efímera nueva, así que **cada llave se usa una sola vez**, y nunca se repite el par llave-nonce. El comentario del código lo explica.

5.4 La geometría: cómo cabe todo en 512 bytes

La celda viaja dentro de un paquete de la mesh. Así se encajan las piezas:



Cada cabecera de 56 = Siguiente(16) + EfímeraSiguiente(32) + MACSiguiente(8). Todas cifradas en capas.

Constante	Valor	De dónde sale
`MAX_CONTENIDO`	504	512 − 8 de encabezado del paquete
`TAM_CUERPO` (celda)	487	504 − 1 (TTL) − 16 (destino)
`TAM_BLOQUE` (R)	446	487 − 41 (versión 1 + efímera 32 + MAC 8)
`TAM_SALTO` (S)	56	16 + 32 + 8
`TAM_SELLADO`	278	446 − 3 × 56
`TAM_PLANO`	230	278 − 48 (sobrecarga de SealedBox)
`MAX_MENSAJE` (anónimo)	**227**	230 − 3 (banderas 1 + largo 2)
`MAX_PUBLICO`	**211**	227 − 16 (tema)
`MAX_MENSAJE_FIRMADO`	**171**	227 − 16 (nombre) − 40 (Box: nonce 24 + tag 16)

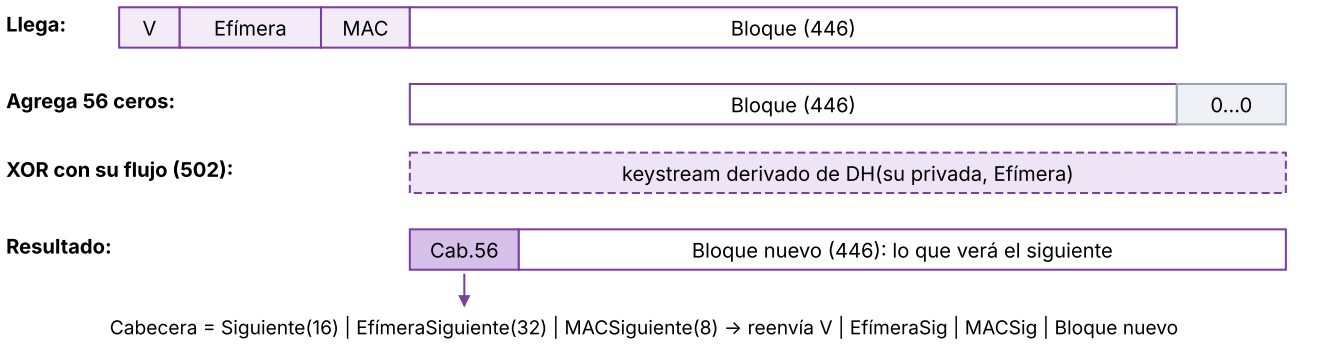
5.5 Cómo arma el emisor la celda (`construir_celda`)

La cebolla se arma **de adentro hacia afuera**: primero lo que verá el último relay, y al final lo que verá el primero.

1. Crea 3 efímeras (una por relay) y, con DH contra la ficha de cada relay, deriva para cada uno su **llave de flujo** y su **llave de MAC**. Genera los 3 flujos de 502 bytes (446 + 56).
2. **Sella** el mensaje para el destino con `SealedBox` → 278 bytes.
3. Arma lo que leerá R3: `nombre del destino(16) | 32 ceros | 8 ceros | sellado` . Los 32 ceros en el lugar de la "efímera siguiente" son la señal de **"tú eres el último"** (nunca es una llave real). Lo cifra con el flujo de R3 y le pega el **relleno determinista** (5.8). Calcula el MAC de R3 sobre eso.
4. Para R2: antepone la cabecera `nombre de R3 | efímera de R3 | MAC de R3` , cifra todo con el flujo de R2, recorta a 446 bytes y calcula el MAC de R2.
5. Igual para R1 con la cabecera de R2. Al final: `versión | efímera de R1 | MAC de R1 | bloque` = 487 bytes. Lo manda por la mesh como paquete CEBOLLA hacia R1.

```python
# cebolla.py (resumido)
sellado = SealedBox(destino).encrypt(armar_plano(mensaje, ...))
interior = nombre16(destino) + CEROS_LLAVE + bytes(8) + sellado
bloque = xor(interior, flujos[-1][:len(interior)]) + relleno_determinista(flujos)
mac = MAC(llave_mac[R3], bloque)
for i in (R2, R1):                        # hacia afuera
    cabecera = nombre16(siguiente) + efimera_pub(siguiente) + mac
    bloque = xor(cabecera + bloque, flujos[i])[:446]
    mac = MAC(llave_mac[i], bloque)
celda = bytes([1]) + efimera_pub(R1) + mac + bloque     # 487 bytes
```

5.6 Qué hace cada relay (`procesar_celda`)



1. Revisa tamaño (487) y versión. Una versión desconocida se descarta sin reenviar.
2. Calcula DH entre **su llave privada de cifrado** y la efímera que viene en la celda, deriva las llaves y **verifica el MAC** sobre el bloque. Prueba con la llave vigente y con las que están en período de gracia (por si rotó hace poco). Si ninguna da un MAC correcto → descarta.
3. **Anti-repetición**: si ya vio esa efímera, descarta. (Solo la recuerda *después* de un MAC válido, para que basura no le llene la memoria.)
4. "Pela" su capa: pega 56 ceros al bloque, hace XOR con su flujo y separa los primeros 56 bytes (la cabecera) del resto (el bloque nuevo, otra vez de 446).
5. Si la efímera siguiente son **32 ceros**, es el último relay: el nombre de la cabecera es el **destino**, y los primeros 278 bytes del resto son el sellado; lo manda como paquete MENSAJE. Si no, reenvía la celda (con la efímera y el MAC del siguiente) como paquete CEBOLLA hacia el siguiente relay.

> **¿Por qué la celda sigue midiendo lo mismo?**
> El relay quita 56 bytes de cabecera por delante, pero agregó 56 ceros por detrás antes de descifrar. Resultado: siempre 446. Un observador que ve la celda entrar y salir de un relay ve dos tiras del mismo tamaño y aparentemente aleatorias, distintas entre sí (no puede emparejarlas por contenido).

5.7 El relleno determinista (la idea de Sphinx)

Este es el punto más sutil del archivo. Cuando R1 pega 56 ceros y aplica su flujo, esos ceros se convierten en 56 bytes "raros" al final del bloque. R2 hace lo mismo y agrega otros 56. Cuando la celda llega a R3, los últimos **112 bytes** (2 × 56) del bloque son esa "basura" generada por R1 y R2.

Problema: el MAC de R3 cubre **todo** el bloque que recibe R3, incluidos esos 112 bytes. El emisor tiene que saber de antemano qué valdrán para poder calcular el MAC. Solución: esos bytes **no son aleatorios**, dependen solo de los flujos de R1 y R2, que el emisor conoce. La función `_relleno_determinista()` simula lo que harán R1 y R2 y obtiene exactamente esos bytes ("filler").

> **¿Y por qué importa?**
> Si el relleno fuera aleatorio y no lo cubriera el MAC, un relay malicioso podría cambiar esos bytes para "marcar" una celda y reconocerla más adelante (ataque de etiquetado). Con relleno determinista **no hay ni un byte** que un relay pueda cambiar sin que el siguiente relay honesto lo detecte. Hay una prueba específica: `test_relay_que_altera_la_celda_es_detectado_por_el_siguiente`.

5.8 El destino abre el sobre: tres tipos de mensaje

El destino recibe un paquete MENSAJE con 278 bytes sellados. Prueba abrirlo con su llave vigente y las de gracia. Dentro hay un "plano" de 230 bytes, siempre del mismo tamaño, que empieza con una **bandera**:

Bandera	Formato del plano	Significado
0 anónimo	`0 | Largo(2) | Texto | ceros`	Nadie sabe quién lo envió, ni el destino. No se puede contestar. Máx. 227 bytes.
1 firmado	`1 | Largo | Remitente(16) | Box(texto) | ceros`	El destino (y solo él) ve el nombre del remitente y lo **verifica** abriendo el `Box` con la llave de la ficha del remitente. Nadie puede suplantarlo. Para contestar, el destino hace otro envío por la cebolla hacia ese nombre. Máx. 171 bytes.
2 público	`2 | Largo | Tema(16) | Texto | ceros`	Para el **gateway**. Nunca lleva remitente. El gateway lo publica en la sala del tema en Cloudflare. Máx. 211 bytes.

Como el plano mide siempre 230 bytes, un mensaje firmado, uno anónimo y uno público **se ven idénticos** desde afuera (hay pruebas que lo comprueban). Un nodo que no es gateway descarta los públicos (`_sin_gateway`); un mensaje privado nunca llega al gancho público.

Si la verificación de un firmado falla, puede ser que el remitente rotó su llave desde la última consulta: el nodo **refresca el directorio** una vez (máximo una consulta cada 10 s, para que mensajes basura no lo obliguen a consultar sin parar) y reintenta.

5.9 Repeticiones

Cada relay guarda en memoria las efímeras ya vistas (10 minutos, máximo 4096). Una celda capturada y reenviada se descarta. En el destino también se recuerda el inicio del sellado: si un relay malicioso entrega dos veces el mismo mensaje, la copia se ignora.

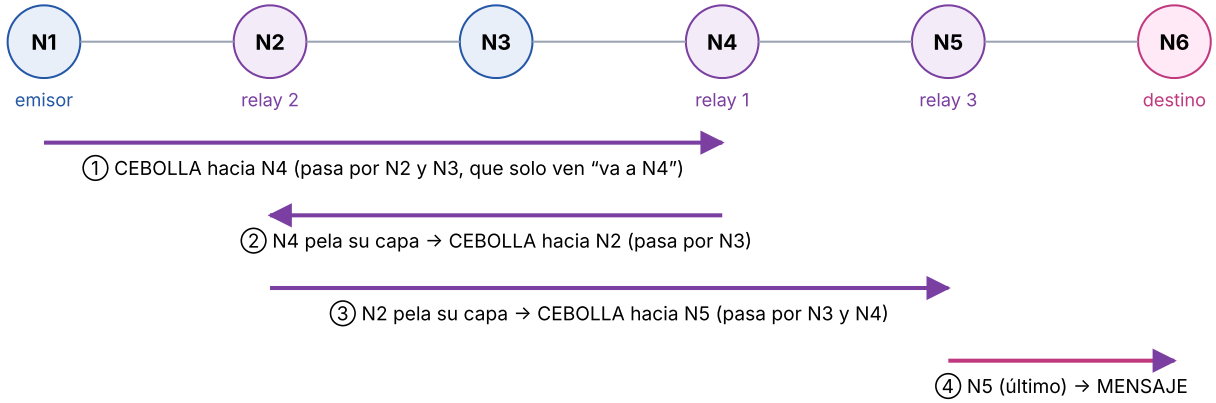
5.10 Rotación de llaves y secreto hacia adelante

La llave de cifrado de cada nodo cambia cada hora (o con `rotarllaves`). La vieja se acepta 10 minutos más (para celdas que ya venían en camino y para quienes tenían la ficha vieja en caché) y luego se **olvida**. Así, si alguien roba el nodo mañana, no puede descifrar celdas capturadas hoy: eso se llama **secreto hacia adelante** (forward secrecy). El código lo llama "parcial" porque dentro de la hora de vida de una misma llave sí se podría descifrar todo lo de esa hora.

La llave de **firma** no rota: si se roba, permite *suplantar* al nodo, pero no descifrar nada. Por eso es la que borra el botón de pánico.

5.11 Ejemplo completo, de punta a punta

Con `scripts/probar_cebolla.ps1` se levanta un directorio y 6 nodos en cadena: **N1 – N2 – N3 – N4 – N5 – N6**. En la terminal de N1 se escribe `anonimo N6 hola`. Supongamos que la ruta elegida al azar es **N4 → N2 → N5**.



N6 abre el sellado e imprime: "[N6] mensaje ANONIMO recibido: hola"

Fíjate en dos niveles distintos:

- **Nivel mesh** (vector de distancias): cada tramo puede cruzar varios nodos normales. Ellos solo leen el destino del paquete (el próximo relay) y lo reenvían.
- **Nivel cebolla**: solo N4, N2 y N5 descifran algo, y cada uno aprende un único nombre.

N3 vio pasar el mensaje cuatro veces, pero cada vez la celda era distinta y no podía saber que era el mismo. N4 recibió la celda de N3: no sabe si N3 la creó o solo la reenvió.

5.12 Limitaciones conocidas (para el RFC)

- **Sin mezcla ni retrasos**: un observador que ve *toda* la red puede correlacionar por tiempos ("entró algo a N4 y 2 ms después salió algo"). Documentado como fuera de alcance.
- **Secreto hacia adelante parcial**: dentro de la ventana de vida de una llave todo se puede descifrar si se roba; no hay un intercambio de llaves por mensaje (como *ntor* en Tor).
- **MAC de 8 bytes** (64 bits) para que todo quepa en 512: suficiente para impedir el marcado de celdas, no para otros usos.
- **De una sola vía**: no hay circuito de regreso. Cada mensaje es independiente; para contestar se hace otro envío.
- El directorio es un punto central: puede **omitir** fichas (no inventarlas). Si omite relays honestos, la elección queda sesgada.
- Python no garantiza borrar memoria: al rotar se sueltan las referencias, que es lo mejor posible en ese lenguaje.

5.13 Las pruebas que lo demuestran (`pruebas/test_cebolla.py` , 48 pruebas)

Corren sobre una mesh falsa en memoria. Algunas, por su nombre: *ningún relay ve origen y destino a la vez · el último relay no ve el origen · todas las celdas miden lo mismo · celdas distintas aunque se mande el mismo mensaje · repetición es descartada · celda alterada no llega · relay que altera la celda es detectado por el siguiente · relay equivocado no puede pelar la celda · llave olvidada ya no abre celdas capturadas antes · celda en camino sobrevive a una rotación · firma suplantada es rechazada · ningún relay ve el remitente del mensaje firmado · mensaje firmado mide igual que el anónimo · mensaje privado nunca llega al gancho público · ningún relay ve el tema ni el texto público.*

PARTE 6 · ÉNFASIS PRINCIPAL

Cloudflare: el edge de FaroNet

Esta es la parte del proyecto que conecta FaroNet con el Internet normal (la "clearnet"). Vive en la carpeta `edge/`, está escrita en TypeScript y corre en **Cloudflare Workers**, en el plan gratuito. Primero vamos a entender qué es todo eso; después, archivo por archivo, qué hace el código.

Resumen rápido: qué pide el enunciado y dónde está

Requisito del enunciado	Archivo	Cómo se cumple
Relay del gateway con firmas	`auth.ts`, `index.ts`, `control.ts`	Ed25519 sobre método, ruta, hora y hash del cuerpo; ventana de 60 s; 409 a repeticiones.
Salas en tiempo real	`sala.ts`	Un DO por tema, WebSockets hibernables, solo lectura.
Feed por tema	`feed.ts`	Atom con XML escapado, autor "anónimo".
Buzón muerto	`buzon.ts`	Blobs cifrados de 1024 bytes, borrar al leer, TTL 24 h.
Directorio público	`directorio.ts`	Réplica verificada ficha por ficha; el gateway no puede falsificar.
Rate limiting	`control.ts`	Ventana fija por hash diario de la IP.
Sin registro / TTL	`wrangler.jsonc`, alarmas	Observability apagada, ninguna IP guardada, borrado real.
Configuración de despliegue	`wrangler.jsonc`	Equivale al `wrangler.toml` que pide el enunciado (formato JSON con comentarios).

6.1 ¿Qué es Cloudflare?

Cloudflare es una empresa con una red enorme de centros de datos repartidos en cientos de ciudades del mundo. Mucha gente la conoce porque se pone "delante" de los sitios web: los protege de ataques, guarda copias para que carguen más rápido y resuelve nombres de dominio. Como sus servidores están físicamente cerca de los usuarios, en el "borde" de la red, a esa ubicación se le llama **edge**.

Además, Cloudflare permite **subir tu propio código** para que corra en esos servidores. Eso es **Cloudflare Workers**.

6.2 Serverless y Workers

Serverless ("sin servidor") no significa que no haya servidores, sino que **tú no administras ninguno**: no instalas un sistema operativo, no abres puertos, no reinicias nada. Solo subes una función y la plataforma la ejecuta cada vez que llega una petición.

> **Analogía**
> Tener un servidor propio es como abrir un restaurante: alquilas el local, compras la cocina y pagas aunque no haya clientes. Un Worker es como darle tu receta a una cadena que tiene cocinas en todas las ciudades: cada vez que alguien pide el plato, la cocina más cercana lo prepara con tu receta.

Un Worker es un módulo que exporta una función `fetch(request, env)`: recibe una petición HTTP y devuelve una respuesta. En FaroNet esa función está al final de `edge/src/index.ts`:

```
export default {
  async fetch(request: Request, env: Env): Promise<Response> {
    // mira la ruta (/sala/..., /buzon/..., /gateway/...) y decide qué hacer
  },
} satisfies ExportedHandler<Env>;
```

- Los Workers corren sobre **V8** (el motor de JavaScript de Chrome) en "aislamientos" muy livianos que arrancan en milisegundos. No es un contenedor ni una máquina virtual.
- `request` es la petición del navegador o del gateway. `env` trae las **vinculaciones** (bindings): las variables de configuración como `GATEWAY_PUBKEY` y los accesos a los Durable Objects (`env.SALA`, `env.BUZON`...).
- Cloudflare agrega a cada petición el encabezado `CF-Connecting-IP` con la IP real del cliente. Es lo único que el Worker de FaroNet lee de la petición, y solo para el límite de peticiones.

> **El problema: un Worker no tiene memoria**
> Cada petición puede ejecutarse en una ciudad distinta, en una copia distinta del código. Una variable global no sirve para guardar los mensajes de una sala: la siguiente petición podría no verla. Hace falta almacenamiento.

6.3 Dónde guardar datos en Cloudflare

Producto	Qué es	¿Lo usa FaroNet?
KV	Almacén clave-valor global; muy rápido para leer, pero los cambios tardan en verse en todas partes (consistencia eventual).	No. Dos lectores podrían recoger el mismo blob del buzón.
D1	Base de datos SQL (SQLite) administrada.	No, pero el SQLite de los Durable Objects es la misma tecnología.
R2	Almacén de archivos (tipo S3).	No.
Durable Objects	Objetos con estado propio, únicos en el mundo y con su base SQLite privada.	**Sí: todo.** Salas, buzón, directorio y control.

El enunciado sugería KV, D1 o R2 para el buzón. El README justifica usar Durable Objects: no hay que crear nada antes de desplegar (`wrangler deploy` basta) y "recoger y borrar" es una sola operación atómica. Si el profesor exige D1 o KV, solo habría que cambiar `buzon.ts`.

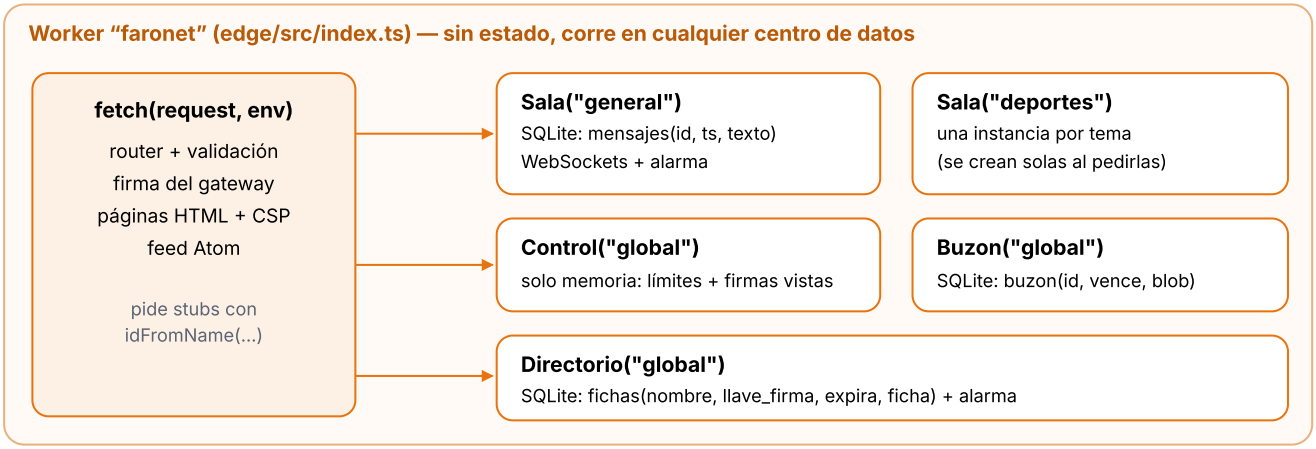
6.4 Durable Objects, explicado con calma

Un **Durable Object** (DO) es una **clase** de TypeScript (por ejemplo `class Sala extends DurableObject`) de la que Cloudflare crea **instancias**. Cada instancia:

- Tiene un **identificador**. Con `idFromName("general")` se obtiene siempre el mismo id para el mismo nombre: todas las peticiones del mundo que pidan la sala `general` llegan a **la misma instancia**.
- Existe en **un solo lugar a la vez** y atiende las peticiones **de una en una** (un solo hilo). Por eso no hay condiciones de carrera: si dos personas recogen el mismo blob al mismo tiempo, una de las dos llega primero.
- Tiene su propio **almacenamiento privado**. En el plan gratuito ese almacenamiento es **SQLite**: se usa con `this.ctx.storage.sql.exec("SELECT ...")`.
- Puede tener **alarmas** y aceptar **WebSockets** (6.5 y 6.6).

El Worker no crea la instancia directamente: pide un **stub** (un "control remoto") y llama sus métodos como si fueran locales. Cloudflare se encarga de llevar la llamada hasta donde vive el objeto (esto se llama RPC):

```
const dir = env.DIRECTORIO.get(env.DIRECTORIO.idFromName("global"));  // stub
const fichas = await dir.listar();                                    // RPC al objeto
return salaDe(env, tema).fetch(request);                               // o reenviarle la petición
```



Los 4 tipos de Durable Object del proyecto. `Control`, `Buzon` y `Directorio` son una sola instancia global; `Sala` tiene una instancia por tema.

`blockConcurrencyWhile(...)`, que aparece en los constructores, le dice a Cloudflare: "no me pases peticiones hasta que termine de crear la tabla". Así nunca se consulta una tabla que todavía no existe.

6.5 WebSockets e hibernación

HTTP normal es pregunta-respuesta: el navegador pide y el servidor contesta. Para un chat en vivo eso no basta: el servidor necesita **avisarle** al navegador cuando llega un mensaje. Un **WebSocket** es una conexión que se queda abierta y por la que el servidor puede enviar datos cuando quiera.

Mantener un objeto despierto solo para sostener conexiones abiertas gastaría "duración" (tiempo de ejecución que cuenta para los límites). La **API de hibernación** resuelve eso: el objeto acepta el socket con `this.ctx.acceptWebSocket(servidor)` y Cloudflare puede sacarlo de memoria mientras las conexiones siguen abiertas. Cuando hace falta (llega un mensaje o se publica algo), lo despierta. `this.ctx.getWebSockets()` devuelve todos los sockets de la sala para reenviarles cada mensaje.

6.6 Alarmas: borrar de verdad

Un Durable Object puede programar una **alarma** con `setAlarm(fecha)`. A esa hora Cloudflare llama al método `alarm()` del objeto, **aunque nadie lo esté usando**. FaroNet lo usa para cumplir los TTL (tiempos de vida) de verdad: no solo esconder lo vencido al consultar, sino **borrarlo de la base**. La alarma se programa para cuando vence el dato más viejo; al sonar, borra lo vencido y se reprograma para el siguiente.

6.7 Wrangler y la configuración

Wrangler es la herramienta de línea de comandos de Cloudflare para Workers. Se instala con npm (está en `edge/package.json`):

Comando	Qué hace
`npx wrangler dev`	Corre el Worker **en tu computadora** (en `http://localhost:8787`), con un simulador del runtime real (workerd/Miniflare), incluidos los Durable Objects.
`npx wrangler login`	Conecta Wrangler con tu cuenta de Cloudflare.
`npx wrangler deploy`	Sube el Worker a Cloudflare y aplica las migraciones. Queda en `https://faronet.<tu-usuario>.workers.dev`.
`npx wrangler types`	Genera `worker-configuration.d.ts`: los tipos de TypeScript del runtime y la interfaz `Env` con tus bindings (por eso ese archivo pesa más de 600 KB: no se edita a mano).
`npx wrangler secret put LIMITE_SAL`	Guarda un secreto en Cloudflare (no queda en el repositorio).

`edge/wrangler.jsonc`, línea por línea

```jsonc
{
  "name": "faronet",                    // nombre del Worker → faronet.USUARIO.workers.dev
  "main": "src/index.ts",               // archivo de entrada
  "compatibility_date": "2026-10-19",   // "compórtate como el runtime de esa fecha"
  "migrations": [                       // historial de clases de Durable Objects
    { "tag": "v1", "new_sqlite_classes": ["Sala"] },
    { "tag": "v2", "new_sqlite_classes": ["Control", "Buzon", "Directorio"] }
  ],
  "durable_objects": { "bindings": [      // nombre en env  →  clase
    { "name": "SALA", "class_name": "Sala" }, ... ] },
  "vars": { "GATEWAY_PUBKEY": "91678aa4…ec24" }, // llave PÚBLICA del gateway
  "observability": { "enabled": false },     // sin logs de invocaciones (privacidad)
  "upload_source_maps": true
}
```

- **Migraciones**: Cloudflare necesita saber cuándo aparece (o se borra) una clase de Durable Object, porque cada una tiene datos guardados. `v1` creó `Sala`; `v2` agregó las otras tres. Se usa `new_sqlite_classes` porque en el plan gratuito los DO deben usar el almacenamiento SQLite.
- **Bindings**: `"name": "SALA"` hace que exista `env.SALA` en el código. Sin esto, el Worker no podría hablar con los objetos.
- **`vars` vs secretos**: `GATEWAY_PUBKEY` es una llave **pública**, así que no pasa nada si está en el repositorio. En local se pone en `edge/.dev.vars` (ignorado por git; hay un `.dev.vars.example`). `LIMITE_SAL`, en cambio, es un secreto opcional y va con `wrangler secret put`.

- `observability: false` : Cloudflare puede guardar registros de cada invocación (con metadatos de las peticiones). Se apaga a propósito: es parte de la política de no registro.

> **¿Qué es la carpeta `edge/.wrangler/` ?**
> Es el estado **local** que crea `wrangler dev` : ahí están los SQLite simulados de cada Durable Object (por ejemplo `.wrangler/state/v3/do/faronet-Sala/…sqlite`). No es lo que está en Cloudflare, no se sube (está en `.gitignore`) y se puede borrar para empezar de cero.

6.8 Las rutas del Worker

Ruta	Quién la usa	Qué hace	Límite
`POST /gateway/publicar`	gateway (firmado)	Publica `{tema, texto}` en la sala del tema.	120/min
`POST /gateway/directorio`	gateway (firmado)	Envía hasta 64 fichas del directorio de la mesh.	120/min
`GET /`	navegador	Página de inicio: abrir una sala por tema.	—
`GET /sala/:tema`	navegador	Página de la sala en vivo.	120/min
`GET /sala/:tema/ws`	navegador	WebSocket de la sala (solo lectura).	120/min
`GET /sala/:tema/historial`	cualquiera	Últimos 50 mensajes en JSON.	120/min
`GET /sala/:tema/feed.atom`	lector de feeds	Feed Atom del tema.	120/min
`GET /directorio[/:nombre]`	cualquiera	Réplica pública del directorio, con firmas verificables.	120/min
`PUT /buzon/:id`	cualquiera	Deposita un blob cifrado de exactamente 1024 bytes.	30/min
`GET /buzon/:id`	cualquiera	Recoge el blob y lo borra.	30/min

Reglas de formato: un tema son 1 a 16 caracteres `a-z 0-9 _ -` (cabe en los 16 bytes de la cebolla); un id de buzón son 32 caracteres hex; un texto público, máximo 300 caracteres.

6.9 El recorrido de una petición (`index.ts`)

`index.ts` es el **router**: parte la ruta en pedazos (`/sala/general/ws` → `["sala","general","ws"]`) y decide la rama. Todas las ramas siguen el mismo patrón: **método correcto → límite de peticiones → validar → (firma) → Durable Object → respuesta con cabeceras seguras**.



En rojo, el código de error que se devuelve si ese paso falla.

Código	Significado en este Worker
200 / 201 / 204	OK / blob depositado / mensaje publicado (sin cuerpo).
400	Datos inválidos: tema, texto, JSON, id o tamaño del blob incorrecto.
401	Firma del gateway ausente, inválida o con hora fuera de la ventana.
404	No existe (ruta, ficha o blob; también un blob ya recogido o vencido).
405	Método HTTP no permitido para esa ruta.
409	Petición del gateway repetida, o ese id de buzón ya tiene un blob.
413	Cuerpo demasiado grande (se corta la lectura apenas pasa el máximo).
426 / 429 / 503	Se esperaba WebSocket / demasiadas peticiones / sala o buzón lleno.

`leerCuerpo()` no confía en el encabezado `Content-Length`: lee el cuerpo por pedazos y **se detiene** en cuanto pasa del máximo (2 KB para publicar, 32 KB para el directorio, 1024 bytes para el buzón). Así nadie puede obligar al Worker a cargar un archivo gigante en memoria.

6.10 Autenticación del gateway (`auth.ts`)

Cualquiera en Internet puede mandarle peticiones al Worker. ¿Cómo sabe el Worker que un `POST /gateway/publicar` viene del gateway de verdad? Con una **firma Ed25519**. El gateway tiene una llave privada propia (`~/.faronet/gateway.llave`, distinta de la del nodo) y el Worker conoce la pública (`GATEWAY_PUBKEY`).

Lo que el gateway firma es este texto de 4 líneas:

```
POST
/gateway/publicar
1793040000                                ← marca de tiempo (segundos Unix)
3a7bd3e2360a3d29eea436fcfb7e44c735d117c4…   ← SHA-256 del cuerpo, en hex
```

y lo manda en dos encabezados: `X-Faro-Ts` (la marca de tiempo) y `X-Faro-Firma` (64 bytes en hex). El Worker reconstruye el mismo texto y verifica con `crypto.subtle.verify` (WebCrypto).

Elemento firmado	Qué ataque evita
Método y ruta	Reusar una firma de `/gateway/publicar` en `/gateway/directorio` (hay una prueba para eso).
Hash del cuerpo	Cambiar el texto o el tema conservando la firma. Firmar el hash en vez del cuerpo entero mantiene el mensaje firmado corto.
Marca de tiempo (±60 s)	Guardar una petición y repetirla horas después.
Firma recordada 120 s en `Control`	Repetirla **dentro** de los 60 s: la segunda vez da **409**.

Detalles: la llave pública se importa una sola vez y se guarda en caché. Hay un plan B para el nombre antiguo del algoritmo en Workers (`NODE-ED25519`). Como dos mensajes idénticos en el mismo segundo producen la misma firma, `gateway_nodo.py` espera 1,1 s y reintenta con otra marca de tiempo cuando recibe un 409.

6.11 Límite de peticiones y anti-repetición (`control.ts`)

`Control` es un Durable Object global que **solo usa memoria** (nunca escribe a disco). Hace dos cosas:

Límite de peticiones (rate limiting) con ventana fija

Para cada cliente lleva una "cubeta": un contador y la hora en que se reinicia. Cada petición suma 1; si el contador pasa el límite antes de que venza la ventana, la respuesta es **429** con `Retry-After`. Límites por minuto: **120 lecturas**, **30 usos del buzón** y **120 intentos al gateway** (que incluyen los intentos con firma mala, para frenar a quien intente adivinar firmas).

¿Cómo identifica al cliente sin guardar su IP?

```
clave = SHA-256( IP + "|" + fecha_de_hoy + "|" + LIMITE_SAL )[primeros 16 bytes]
```

La IP se convierte en un hash que **cambia cada día** (la fecha va dentro) y que nunca se guarda ni se imprime. A `Control` le llega solo ese hash, nunca una IP. Si hay más de 10 000 claves, borra primero las vencidas y luego las más viejas. Si el limitador falla, el Worker **deja pasar** la petición: se prefiere que el servicio siga funcionando a que se caiga por el limitador.

> **Honestidad del modelo de amenazas**
>
> El hash protege contra abuso, **no contra Cloudflare**: Cloudflare ve la IP en claro, así que calcular el hash le resulta trivial. El README lo dice explícitamente.

6.12 Las salas públicas (`sala.ts`)

Un Durable Object **por tema**. Su tabla SQLite es mínima: `mensajes(id, ts, texto)`. No guarda IPs, encabezados ni autores.

1. **Publicar** (llamado solo por el Worker tras verificar la firma): inserta `(ahora, texto)` , borra lo de más de 24 h, deja solo los últimos 200, programa la alarma y **reenvía el mensaje a todos los WebSockets abiertos** de la sala.
2. **WebSocket**: acepta hasta 200 navegadores a la vez (si no, 503 "sala llena"). Es de **solo lectura**: `webSocketMessage()` está vacío a propósito, lo que escriba un navegador se ignora. Desde Internet nadie puede publicar; solo el gateway.
3. **Historial**: los últimos 50 mensajes vigentes en JSON. **ultimos(n)** sirve al feed.
4. **Alarma**: suena cuando vence el mensaje más viejo (+1 s), borra lo vencido y se reprograma. Aunque nadie vuelva a entrar a la sala, sus mensajes desaparecen a las 24 h.

La página de la sala

El Worker genera el HTML directamente (no hay archivos estáticos). El script de la página: (1) pide `/historial` y pinta los mensajes; (2) abre el WebSocket (`wss://` si la página es HTTPS); (3) agrega cada mensaje nuevo arriba; (4) si se corta, muestra "reconectando…" y reintenta cada 3 s. Los textos se insertan con `createTextNode` , nunca como HTML, así que un mensaje con `<script>` se muestra como texto.

6.13 Seguridad de las páginas: CSP con nonce y cabeceras

Un ataque típico a páginas web es **XSS**: lograr que la página ejecute un script ajeno. La **Content-Security-Policy** (CSP) es un encabezado que le dice al navegador qué puede cargar la página. La de FaroNet:

```
default-src 'none';            ← por defecto, nada
script-src 'nonce-Q2x4…';      ← solo el script que traiga este número de un solo uso
style-src 'unsafe-inline';     ← los estilos de la propia página
connect-src 'self';            ← fetch y WebSocket solo al mismo sitio
base-uri 'none'; form-action 'self';
frame-ancestors 'none'         ← nadie puede meter la página en un iframe
```

El **nonce** es un número aleatorio de 16 bytes que se genera **en cada petición**. El único `<script>` de la página lleva ese mismo nonce; cualquier script inyectado no lo conocería y el navegador lo bloquearía. Además, todas las respuestas llevan `Cache-Control: no-store`, `Referrer-Policy: no-referrer` (al salir de la página no se le dice al otro sitio de dónde vienes) y `X-Content-Type-Options: nosniff`. No se ponen cookies. Hay pruebas automáticas para todo esto.

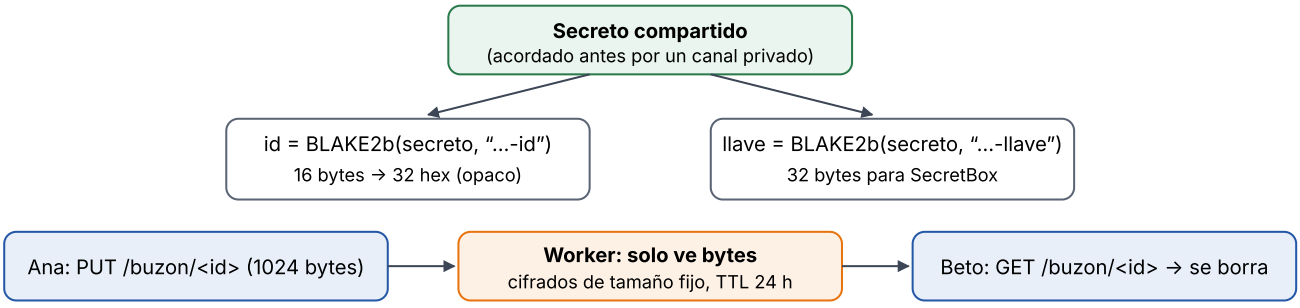
6.14 Los feeds Atom (`feed.ts`)

Un **feed** es un archivo XML con las últimas publicaciones de un sitio, para que programas lectores de feeds avisen cuando hay algo nuevo. Atom es uno de los dos formatos estándar (el otro es RSS). `construirAtom()` es una función pura (sin acceso a Cloudflare, fácil de probar) que arma el XML con las últimas 50 entradas de la sala: cada una con id estable `urn:faronet:sala:<tema>:<id>`, título (los primeros 60 caracteres), fecha, enlace y contenido. El autor es siempre **"anónimo"**: FaroNet de verdad no sabe quién escribió.

`escaparXml()` reemplaza `& < > " '` por sus entidades y elimina caracteres que XML no permite. Sin eso, un mensaje con `</content>` podría romper o manipular el feed (hay una prueba: "el feed escapa el XML"). La página de la sala anuncia el feed con `<link rel="alternate">` para que los lectores lo encuentren solos. Se permite caché pública de 30 s.

6.15 El buzón muerto (`buzon.ts` + `herramientas/buzon.py`)

Un **buzón muerto** (dead drop) viene del espionaje: una persona deja algo escondido en un lugar acordado y otra lo recoge después, sin que se encuentren nunca. Aquí el "lugar" es un id en el Worker y lo que se deja es un blob cifrado.



- **Del lado del cliente** (`buzon.py`): del secreto salen el id y la llave (con BLAKE2b y etiquetas distintas, así uno no revela el otro). El blob es `Nonce(24) | SecretBox(Largo(2) | mensaje | relleno)` = **siempre 1024 bytes** (hasta 982 bytes de texto).
- **Del lado del Worker** (`buzon.ts`): tabla `buzon(id, vence, blob)`. `depositar`: rechaza si el blob no mide 1024 (400), si ese id ya tiene uno vigente (409) o si hay 2000 blobs (503). `recoger`: `DELETE …`

`WHERE id = ? RETURNING blob`, una sola instrucción que **borra y devuelve a la vez**. Como además el Durable Object atiende de uno en uno, dos lectores simultáneos nunca reciben el mismo blob.

- Un id inexistente y uno ya recogido dan el mismo 404: no se filtra si alguna vez hubo algo. Una alarma borra lo vencido a las 24 h.

> **Limitaciones del buzón**
>
> Sin secreto hacia adelante: quien adivine el secreto recoge el blob antes que el destinatario (por eso se pide un secreto largo y al azar, 16+ caracteres). Y hablar directo con el Worker le muestra la IP a Cloudflare: quien deposita y quien recoge quedan enlazables por IP y por tiempo. Mitigación sugerida: hacer esas peticiones desde un nodo detrás de la mesh o desde otra red.

6.16 El directorio público (`directorio.ts`)

Es una **copia de solo lectura** del directorio de la mesh, para que cualquiera en Internet vea qué nodos existen y sus llaves públicas. El gateway (que sí habla con la mesh) le empuja las fichas cada 60 s. La clave del diseño: **el Worker no confía en el gateway**. Cada ficha trae la firma Ed25519 del nodo que la creó, y el Worker la verifica por su cuenta con WebCrypto, aplicando las mismas reglas que `directorio.py`:

- formato y versión correctos; nombre de 1 a 16 bytes UTF-8 sin espacios ni caracteres de control;
- firma válida; no vencida; vencimiento a lo sumo 1 h adelante;
- un nombre no cambia de llave mientras su ficha viva; una ficha más vieja que la registrada se rechaza; máximo 500 fichas.

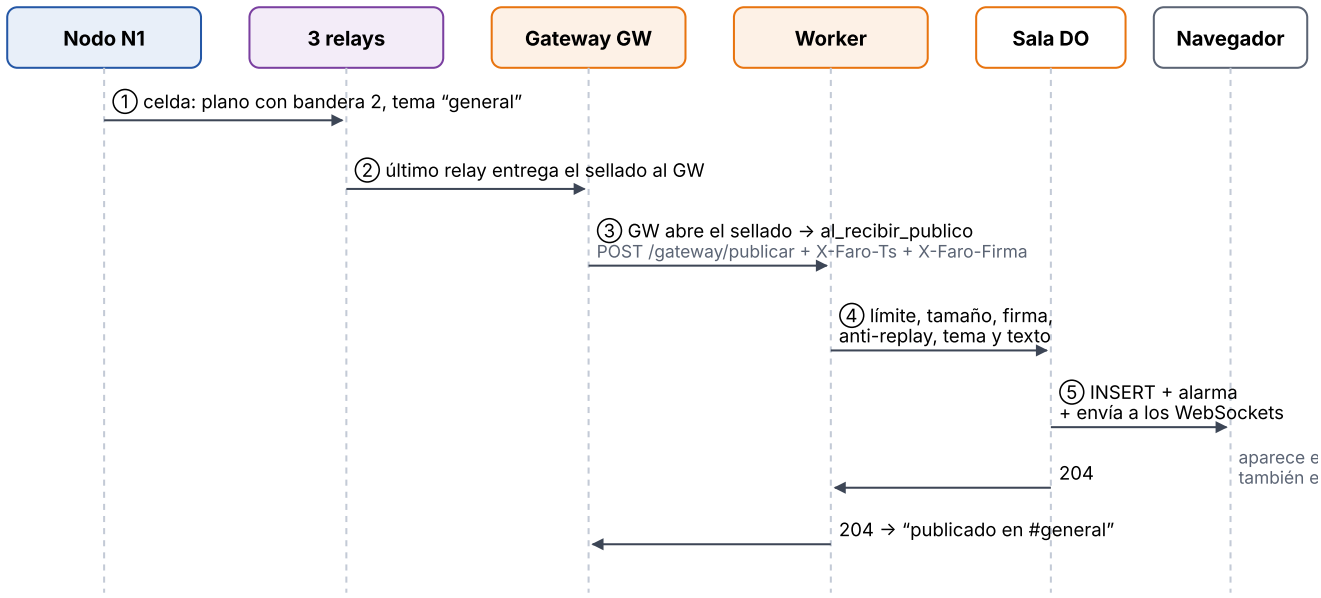
Resultado: un gateway malicioso puede **omitir** fichas, pero **no inventarlas ni alterarlas**. La respuesta pública incluye la ficha cruda en hex (`"ficha"`) para que cualquiera la re-verifique fuera del edge. Lleva `Access-Control-Allow-Origin: *` (cualquier sitio la puede consultar) y caché pública de 15 s. Una alarma borra las fichas vencidas.

6.17 El otro lado: el gateway en Python

Archivo	Qué hace
`herramientas/gateway.py`	Cliente HTTP del Worker. Crea o lee la llave del gateway (`~/.faronet/gateway.llave` , permisos 600), firma exactamente como espera `auth.ts` y ofrece `publicar(tema, texto)` y `publicar_fichas(...)` (en grupos de 64). `--mostrar-llave` imprime la llave pública que va en `GATEWAY_PUBKEY` ; `--probar general "hola"` publica un mensaje de prueba.
`herramientas/gateway_nodo.py`	El gateway **real**: un nodo normal de la mesh que (1) se engancha a `cebolla.al_recibir_publico` y publica en el edge cada mensaje público que le llega por la cebolla, con las mismas validaciones que el Worker (tema, 300 caracteres); y (2) cada 60 s consulta el directorio de la mesh y lo empuja al Worker. No guarda nada a disco ni imprime el contenido, solo “publicado en #tema”.
`herramientas/buzon.py`	Cliente del buzón muerto: `depositar` y `recoger` con un secreto compartido.

6.18 De punta a punta: un mensaje público

Este es el camino completo cuando alguien en la mesh escribe `publico GW general hola desde la cebolla`:



Fíjate en lo que **nadie** aprende: el gateway sabe el texto y el tema, pero recibió el mensaje por la cebolla, así que no sabe quién lo escribió. El Worker y Cloudflare ven la IP del **gateway**, no la del autor. Los relays no vieron ni el tema ni el texto. Y los mensajes privados nunca llegan al gateway: solo la bandera 2 dispara la publicación.

6.19 Política de no registro y lo que Cloudflare igual ve

Lo que el código hace

- Solo lee `CF-Connecting-IP`, y solo para el hash del limitador (en memoria).
- No lee `User-Agent` ni otros encabezados.
- Ninguna tabla guarda IP ni encabezados (hay una prueba que revisa todas las bases).
- `observability` apagada: sin logs de invocaciones.
- TTL reales con alarmas: salas 24 h, buzón 24 h, fichas a su vencimiento.
- Sin cookies; `no-store` y `no-referrer`.

Lo que Cloudflare (el proveedor) igual ve

- La IP de quien se conecta, el User-Agent, tiempos, tamaños y metadatos de TLS.
- Las rutas: qué temas se leen y qué ids de buzón se piden.
- El texto de los mensajes públicos (son públicos por definición) y los blobs cifrados como bytes.
- Todo lo guardado en los Durable Objects.
- La IP del gateway y cuándo publica.

No hay forma de evitarlo desde el código; por eso va en el modelo de amenazas del RFC.

6.20 Probar y desplegar el Worker

Pruebas sin Cloudflare

```
cd edge
npm install
npm run tipos      # revisa los tipos contra el runtime de Workers (tsc)
npm run probar     # 38 pruebas del Worker en Node, sin wrangler ni red
```

`npm run probar` ejecuta el código real de `src/` en Node con un entorno falso: `pruebas/registro.cjs` intercepta `import "cloudflare:workers"` y lo cambia por `falso-cloudflare.cjs`; `pruebas/entorno.ts` le da a cada Durable Object un SQLite en memoria (`node:sqlite`) con la misma interfaz que `ctx.storage.sql`, y simula las alarmas. Comprueba la lógica, **no** el comportamiento real de Cloudflare, y no cubre WebSockets.

Prueba de humo contra un Worker de verdad

```
npx wrangler dev                                  # en otra terminal: http://localhost:8787
npm run humo -- http://localhost:8787
npm run humo -- https://faronet.USUARIO.workers.dev   # el despliegue real
```

`pruebas/humo.mjs` sí cubre lo que falta: CSP de la página de inicio, 401 sin firma, publicar firmado, repetición (409), mensaje **en vivo por WebSocket**, historial, feed, buzón completo y directorio. Firma con `~/.faronet/gateway.llave`, así que su llave pública debe ser la `GATEWAY_PUBKEY` del Worker. Sale con código 1 si algo falla.

Desplegar

1. `py -m faronet.herramientas.gateway --mostrar-llave` → copia los 64 caracteres.
2. Local: ponla en `edge/.dev.vars` como `GATEWAY_PUBKEY=…` y corre `npx wrangler dev`.
3. Producción: pégala en `"vars"` de `wrangler.jsonc`, luego `npx wrangler login` y `npx wrangler deploy` (aplica las migraciones solo). Opcional: `npx wrangler secret put LIMITE_SAL`.
4. Prueba de punta a punta: `py -m faronet.herramientas.gateway --edge <url> --probar general "hola"` y abre `<url>/sala/general` y `/sala/general/feed.atom`.
5. Con la mesh: arranca `gateway_nodo` (`--nombre GW --vecino … --directorio … --edge <url>`) y desde otro nodo escribe `publico GW general hola desde la cebolla`.

> **Ojo con la llave**
>
> La `GATEWAY_PUBKEY` de `wrangler.jsonc` debe corresponder a la llave privada que tenga la máquina donde corre el gateway. Si cambias de computadora o borras `~/.faronet/gateway.llave`, el Worker responderá 401 hasta que actualices la llave pública y vuelvas a desplegar.

6.21 Limitaciones conocidas del edge

- El limitador y el anti-replay viven en la memoria de **un solo** Durable Object: si Cloudflare lo reinicia, se pierden (un replay justo en ese instante cabría) y todo el tráfico pasa por él.
- Cada petición gasta además una petición a `Control`; según el README, el plan gratuito permite 100 000 al día.
- Una firma idéntica en el mismo segundo se toma por repetición (el gateway reintenta).
- `npm run probar` no cubre WebSockets; para eso está `npm run humo`.
- Buzón sin secreto hacia adelante, y enlazable por IP si no se usa desde detrás de la mesh.

PARTE 7

Pruebas automáticas

El proyecto tiene **122 pruebas**: 84 en Python y 38 del Worker. Ninguna necesita hardware ni conexión a Cloudflare.

Archivo	Pruebas	Qué comprueba
`pruebas/test_capa1.py`	18	Trama, CRC, recuperación ante basura y cortes, ACK y reintentos, con puertos serie virtuales.
`pruebas/test_scripts.py`	8	`enviar` y `recibir` de punta a punta sobre puertos virtuales.
`pruebas/test_cebolla.py`	48	Toda la criptografía de la cebolla sobre una mesh falsa en memoria (ver 5.13).
`pruebas/test_gateway.py`	10	Que la firma del gateway coincida con la del Worker, envío de fichas en grupos de 64, y el cliente del buzón (ida y vuelta, tamaño fijo, secreto equivocado…).
`edge/pruebas/edge.test.ts`	38	Firma y replay, salas e historial, feed y escape XML, alarmas, buzón (incluido “dos lectores no reciben el mismo blob”), directorio, rate limiting, ausencia de IPs en las bases, cabeceras y CSP con nonce.

```
# Python (desde la raíz del repositorio; requiere pynacl)
py -m unittest discover -v
# Worker
cd edge && npm run probar
# Mesh completa en Windows: directorio + 6 nodos
.\scripts\probar_cebolla.ps1
```

PARTE 8

Observaciones del repositorio

Al leer todo el código aparecieron algunos detalles que conviene corregir antes de entregar. Ninguno afecta el funcionamiento:

- **La sección "Pendiente" del README principal está desactualizada.** Dice que el gateway y el edge todavía no están, pero ya existen (`edge/` , `gateway.py` , `gateway_nodo.py` , `buzon.py` ; el último commit es "edge: completa el Worker y lo despliega en Cloudflare"). Tampoco aparecen en las tablas del README esos tres archivos ni `test_gateway.py` .
- **Conteo de pruebas**: el README dice 74 pruebas de Python; hoy hay 84 (se sumaron las 10 de `test_gateway.py`).
- `requirements.txt` se menciona en el README y en las instrucciones (`pip install -r requirements.txt`), pero no está en la carpeta. Debería tener `pynacl` y, en la Pi, `pyserial` y `gpiozero` .
- El **Kick-off** dice "JavaScript para el edge"; el Worker quedó en **TypeScript** (que es JavaScript con tipos). Además, `docs/Kickoff.md` está guardado en UTF-16, por lo que algunas herramientas lo muestran con espacios entre letras; conviene guardarlo en UTF-8.
- El enunciado pide `wrangler.toml` ; el proyecto usa `wrangler.jsonc` , que es el mismo archivo de configuración en formato JSON. Vale la pena mencionarlo en la documentación.
- Siguen pendientes, según el README: el **RFC** en ASCII (Capa 1 y FaroNet, con consideraciones de privacidad y seguridad), el esquemático, la BOM, las fotos del Faro soldado y la bitácora del multímetro. Los docstrings de `cebolla.py` , `capa1.py` y el README de `edge/` ya traen mucho material listo para el RFC (formatos, limitaciones y qué ve Cloudflare).

PARTE 9

Preguntas que te pueden hacer

Respuestas cortas para la revisión. Las más largas están en las partes 5 y 6.

¿Por qué todos los paquetes miden 512 bytes?
Porque el tamaño filtra información. Con tamaño fijo, un mensaje anónimo, uno firmado y uno público se ven iguales, y sobre TCP basta leer 512 bytes por paquete.

¿Qué diferencia hay entre el CRC de la Capa 1 y el MAC de la cebolla?
El CRC detecta errores *accidentales* de la luz; cualquiera puede recalcularlo. El MAC necesita una llave secreta: detecta alteraciones *intencionales*.

¿Qué pasa si un relay es malicioso?
Solo conoce a su anterior y a su siguiente. Si altera la celda, el siguiente relay honesto lo detecta por el MAC y la descarta. Si la repite, se descarta por la efímera ya vista. Haría falta que los tres relays estuvieran coludidos para unir origen y destino.

¿Por qué el último relay no puede leer el mensaje?
Porque va sellado con `SealedBox` hacia la llave pública del destino. El último relay solo ve 278 bytes cifrados y el nombre del destino.

¿Cómo sabe el destino quién le escribió en un mensaje firmado, si los relays no lo saben?
El nombre viaja *dentro* del sellado, junto a un `Box` que solo pudo crear quien tiene la llave privada de ese remitente. El destino lo comprueba con la llave pública de la ficha del remitente.

¿Por qué Durable Objects y no KV para el buzón?
Porque "recoger y borrar" tiene que ser atómico. KV es eventualmente consistente: dos lectores podrían recibir el mismo blob. Un DO atiende de uno en uno y `DELETE … RETURNING` hace las dos cosas en una sola instrucción. Además no hay nada que aprovisionar antes de desplegar.

¿Cómo evita el Worker que alguien que no es el gateway publique en las salas?
Las publicaciones exigen firma Ed25519 con la llave del gateway sobre método, ruta, hora y hash del cuerpo. Los WebSockets de los navegadores son de solo lectura.

¿Y si alguien captura una petición del gateway y la repite?
Fuera de los 60 s la marca de tiempo la invalida (401). Dentro de los 60 s, `Control` recuerda la firma y responde 409.

¿El Worker guarda IPs?
No. Solo usa la IP para calcular, en memoria, un hash con la fecha del día para el límite de peticiones. Pero Cloudflare como empresa sí ve las IPs; eso no se puede evitar desde el código y está documentado.

¿Qué impide que el gateway mienta en el directorio público?
Cada ficha está firmada por su nodo y el Worker la verifica. El gateway puede omitir fichas, pero no inventarlas ni modificarlas. Y cualquiera puede re-verificarlas porque se publica la ficha cruda.

¿Qué es el "secreto hacia adelante" y por qué es parcial?
Que robar las llaves hoy no permita leer tráfico de ayer. Las llaves de cifrado rotan cada hora y la vieja se olvida a los 10 min; pero dentro de la vida de una misma llave, su robo sí abriría lo capturado en esa ventana.

¿Por qué TTL de 16 y distancia infinita de 16?
Igual que en RIP: limita los saltos para que un paquete no gire para siempre en un bucle, y define cuándo un destino se considera inalcanzable.

¿Qué hace el botón de pánico?
Detiene el nodo, sobrescribe con ceros y borra las llaves de firma, rota la dirección física del Faro y lo indica con los LEDs. Las llaves de cifrado nunca estuvieron en disco.

PARTE 10

Glosario

Término	Significado
ACK	Confirmación de que una trama llegó bien.
Alarma (DO)	Llamada programada que Cloudflare hace a un Durable Object en una fecha dada, aunque nadie lo use.
Atom	Formato XML estándar de feeds.
Baudios	Símbolos por segundo del puerto serie; aquí, aproximadamente bits por segundo.
Bellman-Ford	Algoritmo de caminos más cortos que usa el vector de distancias.
BER	Tasa de error de bits: bits errados / bits enviados.
Binding	Conexión declarada en `wrangler.jsonc` que aparece en `env` (variables, Durable Objects…).
BLAKE2b / SHA-256	Funciones de hash. BLAKE2b admite llave (sirve como MAC y para derivar llaves).
Celda	Unidad de la cebolla: 487 bytes = versión, efímera, MAC y bloque cifrado.
Clearnet	El Internet normal, fuera de FaroNet.
CORS	Reglas del navegador sobre qué sitios pueden leer una respuesta; `*` = cualquiera.
CRC-16	Suma de verificación para detectar errores accidentales en una trama.
CSP / nonce	Política que limita qué puede cargar una página; el nonce es un valor de un solo uso que autoriza un script concreto.
Dead drop / buzón muerto	Lugar acordado donde uno deja algo y otro lo recoge sin encontrarse.
DH (Diffie-Hellman), X25519	Forma de que dos partes obtengan un secreto común sin enviarlo.
Durable Object	Objeto con estado de Cloudflare, único por id, de un solo hilo, con SQLite propio.
Edge	Los servidores del “borde” de la red, cerca de los usuarios.
Ed25519	Algoritmo de firma digital.
Efímera (llave)	Llave creada para un solo uso y descartada.
Ficha	Entrada firmada del directorio: nombre, llaves públicas, vencimiento y firma.
Filler / relleno determinista	Bytes de la cola del bloque que el emisor precalcula para que el MAC de cada salto los cubra.
Forward secrecy	Secreto hacia adelante: robar llaves hoy no abre tráfico pasado.
Gateway	Nodo que saca a Cloudflare solo los mensajes públicos.
Hibernación (WebSocket)	El DO puede salir de memoria mientras sus conexiones siguen abiertas.

idFromName	Convierte un nombre en el id de un Durable Object: mismo nombre, misma instancia.
MAC	Código de autenticación: hash con llave que detecta alteraciones.
Mesh	Red en malla, sin centro, donde los nodos reenvían por otros.
Migración (Wrangler)	Registro de qué clases de Durable Object se crean o borran.
Rate limiting	Límite de peticiones por cliente en una ventana de tiempo.
Relay	Nodo intermedio de la cebolla que pela una capa y reenvía.
Replay	Ataque de repetir un mensaje válido capturado.
RIP / vector de distancias	Protocolo de rutas donde cada nodo anuncia “llego a X en n saltos”.
RPC / stub	Llamar un método de un objeto remoto como si fuera local, mediante un “control remoto”.
SealedBox / Box / SecretBox	Cifrados de PyNaCl: hacia una llave pública (anónimo), entre dos pares de llaves (autenticado), y con llave compartida.
Serverless	Modelo en que subes funciones y la plataforma las ejecuta; no administras servidores.
Trama	Unidad de la Capa 1 (hasta 128 bytes) con sincronía, encabezado, datos y CRC.
TTL	Tiempo o saltos de vida: de un paquete en la mesh, o de un dato guardado en el edge.
WebSocket	Conexión que se queda abierta y permite al servidor enviar datos en cualquier momento.
Worker	Función que corre en la red de Cloudflare ante cada petición HTTP.
Wrangler	Herramienta de terminal para desarrollar y desplegar Workers.
XOR	Operación bit a bit que, aplicada dos veces con la misma llave, devuelve el original.